UNIVERSITÉ DE GENÈVE

PROJET 8 :

EXTRACTION AUTOMATIQUE DE STRUCTURES DE MACROMOLÉCULES BIOLOGIQUES

RAPPORT

SUPERVISEUR : MME FRÉDÉRIQUE LISACEK

EQUIPE : LAYAL SALEH, RANIA ELHADJ, SINA DIVINE ABINA

1 Identification des besoins

De nos jours, la bio-informatique est devenue un pilier essentiel de la biologie moléculaire, une discipline qui ne peut plus se passer d'outils informatiques, par exemple pour l'étude du génome. Toutefois, il s'agit d'un domaine très vaste, qui comprend de nombreuses sous-disciplines, chacune ayant ses propres besoins. Le monde vivant est en effet défini par plusieurs niveaux d'organisation et d'analyse, relevant de domaines spécifiques de la biologie, comme l'étude des protéines. Les outils informatiques modernes, notamment l'intelligence artificielle, ont permis de nombreux progrès : la modélisation et l'étude des fonctions des protéines se sont considérablement développées grâce à ces technologies, tout comme dans de nombreux autres domaines de la biologie.

Dans ce contexte, GlyConnect est une base de données biologique, particulièrement riche en informations sur les glycoprotéines. C'est avec Mme Lisacek, notre superviseure, directrice du Proteome Informatics Group au SIB, et l'une des responsables de GlyConnect que nous avons travaillé durant les trois derniers mois. Mme Lisacek nous a présenté les objectifs du projet ainsi que les bases biologiques nécessaires à sa compréhension, afin de nous permettre de mieux appréhender les objets manipulés et les outils utilisés.

GlyConnect est particulièrement utile pour les étudiants, les doctorants et les chercheurs en biologie, notamment parce qu'il contient des informations biologiques détaillées sur les protéines, telles que leurs sites de glycosylation, leurs structures tridimensionnelles ainsi que les informations propres à leurs glycanes (voir explications plus bas). GlyConnect est accessible au public, directement depuis Internet mais aussi à travers son API. Elle dépend de deux bases de données de référence : (1) UniProt (www.uniprot.org) pour les protéines et (2) PDB (https://www.rcsb.org) pour les structures tridimensionnelles de protéines.

Notre mission était d'enrichir davantage cette base de données avec des structures 3D supplémentaires pour les protéines déjà présentes. Les protéines sont des macromolécules essentielles formées par l'assemblage de chaînes d'acides aminés. Les glycoprotéines possèdent ce qu'on appelle des glycosylations, ou « décorations » : des configurations spécifiques de glycanes (ou sucres), de composition chimique particulière, qui se placent à des endroits précis sur la protéine, avec une grande variabilité. Ces glycosylations peuvent être représentées et visualisées en trois dimensions. La base de données de GlyConnect contient des milliers de glycoprotéines (environ 2 700 au total), cependant les données expérimentales de structures tridimensionnelles sont manquantes pour un grand nombre d'entre elles. Ces données manquantes peuvent être remplacées par des modèles mais les logiciels de modélisation n'intègrent pas les "décorations". Ainsi, nous nous sommes chargées d'une extension de cette base de données afin que le

plus grand nombre de protéines dispose d'au moins un modèle 3D incluant les glycanes.

Pour ce faire, nous avons récupéré des modélisations depuis un site complémentaire, GlycoShape (https://glycoshape.org), qui partage les références à UniProt et PDB mais dont la base de données est centrée sur les glycanes. Un outil associé, nommé Re-Glyco, accessible en ligne, permet de simuler la glycosylation sur une protéine. Nous avons effectué des requêtes via leur API pour lancer Re-Glyco, afin de récupérer les structures 3D glycosylées d'une liste de protéines sous forme de fichiers .pdb (le format standard proposé par PDB). Au début du projet, l'objectif initial était de récupérer les configurations 3D uniquement pour les protéines n'en possédant aucune. Cependant, à la suite des discussions avec notre superviseure Mme Lisacek, la portée du projet a été élargie :

- Récupérer les structures 3D glycosylées pour les protéines de GlyConnect via l'API Re-Glyco
- Traiter l'ensemble des protéines de la base de données (~ 2400), même celles disposant déjà d'une structure, puisque celles-ci seront dans tous les cas différentes de celles récupérées (ie. cela ne fait qu'enrichir la base).

Dans ce cadre, le script développé dans ce projet s'adresse principalement aux membres de l'équipe de développement de GlyConnect, c'est-à-dire des développeurs ou bioinformaticiens souhaitant enrichir la base de données de manière automatisée. Il peut également être utilisé par tout développeur souhaitant extraire des structures glycosylées depuis GlycoShape pour une autre base de données partageant les mêmes références (UniProt, PDB).

2 Développement

2.1 Conception

Scénario d'utilisation :

Un scénario typique d'utilisation du code sera le suivant :

- Un membre de l'équipe de développement de la base de donnéess utilisateur souhaite enrichir la base GlyConnect avec des structures protéiques glycosylées en 3D.
- Il dispose d'une liste d'identifiants UniProt (par exemple 2 400 protéines).
- Il lance le script principal (enrich_glyconnect.py) en indiquant le dossier de sauvegarde et la liste en entrée. Il peut aussi modifier la liste des Glycans par défaut si il le souhaite
- Le script interroge l'API GlycoShape (Re-Glyco) pour chaque protéine : création d'une session, soumission d'un job, interrogation périodique du statut, puis téléchargement du fichier PDB glycosylé si disponible.

- Les structures récupérées sont sauvegardées localement sous forme de fichiers .pdb, nommés d'après l'identifiant UniProt.
- Un rapport JSON (batch_report.json) récapitule l'état de chaque protéine (ALREADY_DONE, ERROR/API_ERROR, ERROR/NO_PDB).
- L'utilisateur peut ensuite charger les fichiers .pdb dans un visualiseur 3D (MolStar) pour vérifier la qualité des glycosylations.

Ce code peut aussi être utilisé pour enrichir n'importe quelle autre base de données qui partagent les références à UniProt et PDB ou simplement pour extraire des protéines de ReGlyco. Tout développeur souhaitant développer une base de données à partir de GlycoShape pourra l'utiliser aussi bien pour extraire 1 seule protéine ou plusieurs (pour les détails de l'utilisation se référer au README.md).

Contraintes du Programme :

Le travail étant d'enrichir une base de données et donc de sortir près de 2000 fichiers .pdb, on devait prendre en compte plusieurs subtilités (Comment agir si une protéine échoue au milieu ? Comment ne pas surcharger l'API etc.). Ainsi, le programme doit satisfaire les contraintes suivantes :

- **Volume** : ~800 protéines à traiter séquentiellement via une API tierce non contrôlée (on ne connait pas les limites de l'API)
- **Résilience** : les jobs peuvent échouer pour des raisons hétérogènes (protéine absente d'AlphaFold*, clash stérique lors du placement de glycanes, timeout serveur)
- **Reprises** : le script doit pouvoir être relancé sans retraiter les protéines déjà complétées
- **Traçabilité** : chaque résultat (succès ou erreur) doit être logué de façon structurée

*AlphaFold : logiciel qui utilise l'Intelligence Artificiel pour prédire les structures 3D des protéines à partir de leurs séquences d'acides aminés

Structures de données :

Les requêtes sur les protéines se fait en plusieurs étapes selon les modalités de l'API utilisé, pour ReGlyco, le pipeline de notre code est donc :
Insérer la liste de protéines lancer le job suivre le statut récupérer les fichiers

Ainsi, tout au long de ce processus, on a dû manipuler des entités différentes, en entrées, en sortie mais aussi à l'intérieur du code lui-même.

1. **Données en entrée**

- Un fichier .txt contenant les identifiants UniProt des protéines à traiter, à raison d'un identifiant par ligne

- Une liste noire statique PROTEINS_IGNORE contenant les protéines identifiées comme définitivement non traitables (type O-GlcNAc, non supporté par Re-Glyco)

2. **Données échangées avec l'API**

À chaque protéine, le script effectue plusieurs échanges successifs avec l'API :

- POST /api/sessions → reçoit une session_uuid et les métadonnées de la protéine (sites de glycosylation détectés)
- POST /api/jobs → reçoit un job_uuid permettant de suivre l'état du traitement
- GET /api/jobs/{uuid} → reçoit un statut (queued, leased, running, completed, failed) — interrogé toutes les 5 secondes
- GET /api/jobs/{uuid}/files reçoit la liste des fichiers disponibles
- GET /api/jobs/{uuid}/files/all.pdb → télécharge le fichier .pdb binaire

3. **Données en sortie**

Le tableau suivant récapitule les fichiers produits par le script :

Fichier	**Format**	**Contenu**
{uniprot_id}.pdb	Binaire	Structure 3D de la protéine avec glycanes attachés
batch_report.json	JSON	Rapport détaillé par protéine (statut, erreur, taille)
batch_report.csv	CSV	Tableau récapitulatif lisible
succes.txt	Texte	Liste des protéines traitées avec succès + taille KB
erreurs.txt	Texte	Détail des erreurs par protéine (type + message)
proteins_failed.txt	Texte	Liste des protéines en erreur, prête à relancer

Chaque protéine traitée produit également un enregistrement intermédiaire protein_result (dictionnaire Python) accumulé dans la liste summary, sérialisée en batch_report.json à la fin de l'exécution de la forme :


```
{
  "protein_id": "P22105",
  "status": "ALREADY_DONE",
  "error_type": null,
  "error_message": null,
  "pdb_saved": true
},
```

Cas d’utilisation identifiés :
Voici quelques cas d’utilisations qu’on a identifié :

Use-case	Acteur	Résultats attendus
Téléchargement réussi	Pipeline automatique	Session créée, job soumis, statut polled jusqu'à completed, all.pdb téléchargé et sauvegardé → statut SUCCESS
Protéine déjà traitée	Pipeline automatique	os.path.exists() détecte le .pdb existant → statut ALREADY_DONE, aucun appel API, continuation immédiate
Protéine absente d'AlphaFold	API GlycoShape	POST /api/sessions échoue (HTTP 500) → exception API_ERROR → statut ERROR, consignée dans erreurs.txt et proteins_failed.txt
Absence de structure glycosylée	Re-Glyco (API)	Job completed mais all.pdb absent de la liste → exception NO_PDB → statut ERROR, retentable via proteins_failed.txt
Délai dépassé	Pipeline automatique	Job non terminé après 300 secondes (60 appels × 5s) → exception TIMEOUT → statut ERROR
Interruption manuelle	Utilisateur (Ctrl+C)	KeyboardInterrupt intercepté → finally ferme le CSV → JSON et proteins_failed.txt écrits avec les résultats partiels
Visualisation 3D	Utilisateur	Chargement du .pdb dans MolStar, affichage glycane + protéine

2.2 Implémentation

Lors de l'implémentation du code on a dû faire plusieurs choix, ceux-ci étaient dictés par les contraintes identifiées avant mais aussi par les observations lors de la manipulation des grandes listes de protéines. Ainsi, voici les choix importants que nous avons réalisés :

- **Architecture séquentielle avec skip intelligent** : à chaque itération, le script vérifie si le .pdb existe déjà sur disque et passe à la suivante si c'est le cas. Cela permet de relancer le script sans dupliquer le travail.
- **Classification des erreurs par préfixe** : chaque fonction wrappée lève une exception avec un préfixe standardisé (API_ERROR:, NO_PDB:, etc.), permettant à la fonction classifier_erreur() de les catégoriser proprement sans parsing complexe.
- **Persistance incrémentale** : le CSV est flushed après chaque protéine (csv_file.flush()) et le JSON est sauvegardé en finally, garantissant qu'une interruption (Ctrl+C ou crash) ne perd pas les résultats déjà collectés.
- **Blacklist statique** : les protéines identifiées comme définitivement non traitables (timeout répété, fichiers trop volumineux) sont exclues via PROTEINS_IGNORE avant la boucle principale.
- **Traitement séquentiel avec pause** : les protéines sont traitées une par une avec un délai de 2 secondes entre chaque requête, et de 5 secondes entre chaque vérification de statut. Cela évite de saturer l'API dont les limites ne sont pas documentées.

Outils utilisés :

Les principaux outils utilisés sont :

- Python : Langage principal choisi pour sa lisibilité, sa rapidité de prototypage et son écosystème qui facilite la manipulation des API, on n'a juste qu'à importer les librairies nécessaires :
    - Requests : Bibliothèque tierce python pour les requêtes http, utilisé pour interagir avec l'API Glycoshape ReGlyco.
    - On a aussi utilisé d'autres librairies pour nos sorties et notre manipulation internes (time, json, csv et os).

- API Glycoshape Re-Glyco (glycoshape.org) : Outil central du projet

C'est un pipeline de prédiction et d'attachement de glycanes sur des structures protéiques 3D.

- JSON/CSV : Formats lisibles et interopérables ; utilisés pour la sérialisation des rapports et les formats lisibles
- MolStar : Standard industriel pour structures macromoléculaires

Utilisé pour la visualisation 3D finale

2.3 Tests et évaluation

Méthodes de tests utilisées :

Les tests ont été conduits de manière progressive et itérative.
Dans un premier temps, le pipeline a été validé sur une seule protéine (par exemple P15328).

La validation visuelle a ensuite été effectuée en comparant les structures générées dans le code avec des structures de référence qu'on a sorti manuellement de GlycoShape (exemple : 4KMX) via le visualiseur Mol*.
Le pipeline a ensuite été généralisé à une liste de 15 protéines fournie par Mme Lisacek, puis étendu progressivement aux 2400 protéines réparties entre les membres de l'équipe.

Critères de validation :

Le logiciel est considéré comme fonctionnel pour une protéine donnée si les critères suivants sont réunis : la session est créée avec succès (HTTP 201) et le job passe à l'état "completed". Ces vérifications sont effectuées automatiquement dans le code à chaque itération.

Types d'erreurs identifiés :

Deux types d'erreurs ont été observés et documentés dans le CSV final :

- API_ERROR 422 : correspond à des protéines pour lesquelles aucune structure ni dans PDB ni dans AlphaFold.
- NO_PDB : correspond à des protéines dont la structure existe mais qui ne présentent aucun site de glycosylation détecté par Re-Glyco.

Ces deux types d'erreurs constituent des limitations biologiques inhérentes aux données, et non des erreurs de code.

Amélioration :

Deux améliorations sont envisageables pour les itérations futures du projet.
La première consiste à diversifier les sources de données.
Pour les protéines non couvertes par GlycoShape, il serait pertinent d'interroger directement d'autres bases (PDB, AlphaFold) afin de récupérer les structures manquantes par d'autres voies.

Une seconde amélioration concerne la validation des résultats. Dans la version actuelle, le script vérifie uniquement que le fichier PDB a été généré et téléchargé, mais ne contrôle pas son contenu. Une étape de validation supplémentaire (visuelle) serait un plus.

3 Organisation

Division du travail :
Deux acteurs principaux ont contribué au projet.

Mme Frédérique Lisacek, en tant que cliente et superviseure, a défini les objectifs biologiques et assuré la validation et l'orientation tout au long du projet.

Au sein de l'équipe, le travail a été organisé en deux phases :

Dans la phase d'initiation, l'équipe a consacré les deux premières semaines à comprendre qu'est-ce que c'est une API et comment fonctionne ReGlyco.
Ensuite, chaque membre a testé individuellement le pipeline sur une protéine puis sur la liste de 15 protéines.

Dans la phase de programmation, on a réparti la liste des 2400 protéines en trois sous-listes de 800 protéines, chacune prenant en charge sa portion.
Chaque membre a d'abord développé une version primaire du script, couvrant l'ensemble du pipeline : création de session, soumission du job, attente du statut, téléchargement du fichier PDB et gestion des erreurs. Les approches ayant différé sur certains points par exemple : la structure du code, la gestion des cas particuliers et le format des fichiers de sortie.
Les meilleures pratiques de chaque version ont été identifiées et fusionnées pour produire le script final commun.
Finalement, M. Ojas Singh, développeur de GlycoShape, est intervenu en support technique pour la vérification du code et la rectification de certaines erreurs liées à l'utilisation de l'API.

D'autre part, la rédaction des procès-verbaux a fait l'objet d'une rotation entre les membres à chaque réunion.

Suivi du travail :

Sept réunions ont été tenues entre le 03/03/2026 et le 21/05/2026, en alternant entre format Zoom et présentiel. Chaque réunion se concluait par la définition d'objectifs à atteindre pour la séance suivante, permettant un avancement régulier du projet.

Un procès-verbal a été rédigé pour chaque réunion, documentant les décisions prises et les tâches à effectuer.

Volume horaire :
Le volume horaire total individuel se décompose comme suit :

- 5 heures pour les réunions avec la superviseure.

- 4 heures pour la rédaction des procès-verbaux
- 10 heures pour les rendez-vous d'équipe
- 30 heures de travail personnel pour chaque membre : compréhension de l'API, code, etc...

Sur la durée totale du projet (environ 10 semaines), cela représente un volume cumulé d'environ 49 heures par étudiant (147 pour toute l'équipe).

4 Projet

Correspondance avec le produit demandé :

L'objectif principal du projet a été atteint. Le pipeline produit des fichiers exploitables pour la grande majorité des protéines traitées et pourra servir à enrichir la base de données GlyConnect.

Difficulté technique :

La difficulté technique était adaptée à notre niveau, tout en restant stimulante. La principale difficulté ne résidait pas dans la complexité algorithmique du code, mais dans la compréhension du comportement réel de l'API GlycoShape, dont la documentation incomplète a nécessité une phase d'exploration et de tests successifs. La communication avec M. Ojas Singh, développeur de GlycoShape, a été déterminante pour débloquer certaines situations.

Cette expérience nous a permis de réaliser l'importance de tester progressivement (d'abord une protéine, puis 15, puis 800) avant de généraliser à grande échelle, et de la valeur d'une gestion d'erreurs rigoureuse dans un script batch.

5 Annexes

La documentation utilisateur (installation et utilisation du script) est disponible dans le fichier README.md du dépôt GitLab. Le code source complet est accessible dans le même dépôt sous le fichier enrich_glyconnect.py.